

Text-to-SQL Assistant

with LangChain & Gradio

Data Science Program

Student Project Brief

May 2026

Project Overview

This project challenges you to build a real-world **Text-to-SQL application** using **LangChain** and **Gradio**. Your app will accept plain-English questions, convert them into SQL queries with a Large Language Model (LLM), execute those queries against a relational database, and present the results through a clean web interface — all without the user needing to know SQL.

Every team works on the **same dataset** (the Chinook Music Store database, described in detail below). Your **implementation, prompt design, pipeline architecture, and UI** are entirely your own.

What Is Text-to-SQL?

Text-to-SQL is the task of automatically translating a natural language question into a valid SQL query. Instead of a user writing:

```
SELECT a.Name, COUNT(t.TrackId) AS Tracks FROM Artist a JOIN Album al ON
a.ArtistId=al.ArtistId JOIN Track t ON al.AlbumId=t.AlbumId GROUP BY a.ArtistId ORDER BY
Tracks DESC LIMIT 10;
```

...they simply ask: *"Which 10 artists have the most tracks?"* Your app does the rest.

Why does this matter?

- Most domain experts and analysts know their data but **cannot write SQL**.
- Text-to-SQL bridges human language and structured data — one of the fastest-growing LLM use cases at companies like Google, Microsoft, Meta, and Databricks.
- It showcases the full LLM pipeline: **prompt engineering** → **LLM** → **SQL** → **database** → **results** → **UI**.

The Dataset — Chinook Music Store (SQLite)

All teams use the **Chinook database**, a well-known open-source sample database that models a digital music store (think: a simplified iTunes).

Property	Detail
Format	SQLite — a single .sqlite file, no database server required
Tables	11 tables
Records	~347 artists · ~347 albums · 3,503 tracks · 59 customers · 412 invoices
Date range	Sales data spanning 2009 – 2013
Source	github.com/lerocha/chinook-database (2.5k stars, MIT-compatible)
Download	Automatic — your notebook fetches it for you on first run

Why Chinook?

Reason	Detail
Realistic relationships	Multiple foreign keys across all tables — great for JOIN-heavy questions
Rich question variety	Covers sales, inventory, customer, employee, and music-domain questions
Widely documented	Easy to find schema info, benchmark queries, and examples online
Compact but complete	Small enough to run on a laptop, large enough to be interesting
Industry standard	Used in demos by SQLAlchemy, LangChain, DBever, and many ORM tools

Full Schema — All 11 Tables

Study this carefully. Your LLM prompt must describe the schema accurately or the model will hallucinate column names and produce wrong SQL.

Music domain tables

Table	Key Columns	Description
Artist	ArtistId (PK), Name	347 artists (AC/DC, Beatles, Miles Davis, ...)
Album	AlbumId (PK), Title, ArtistId (FK)	347 albums linked to artists
Track	TrackId (PK), Name, AlbumId (FK), MediaTypeId (FK), GenreId (FK), Composer, Milliseconds, Bytes, UnitPrice	3,503 tracks with genre, duration, price
Genre	GenreId (PK), Name	25 genres — Rock, Jazz, Metal, Pop, Classical, ...
MediaType	MediaTypeId (PK), Name	5 types — MPEG, AAC, Protected AAC, ...
Playlist	PlaylistId (PK), Name	18 curated playlists
PlaylistTrack	PlaylistId (FK), TrackId (FK)	Many-to-many join: playlists ↔ tracks

Business domain tables

Table	Key Columns	Description
Customer	CustomerId (PK), FirstName, LastName, Company, Address, City, State, Country, Email, SupportRepId (FK)	59 customers worldwide with support rep link
Employee	EmployeeId (PK), LastName, FirstName, Title, ReportsTo (FK), BirthDate, HireDate, City, Country, Email	8 employees with org-chart hierarchy (ReportsTo)
Invoice	InvoiceId (PK), CustomerId (FK), InvoiceDate, BillingCity, BillingCountry, Total	412 invoices (2009–2013)
InvoiceLine	InvoiceLineId (PK), InvoiceId (FK), TrackId (FK), UnitPrice, Quantity	2,240 line items — what was purchased on each invoice

Entity Relationships (summary)

- One **Artist** has many **Albums**
- One **Album** has many **Tracks**
- Each **Track** belongs to one **Genre** and one **MediaType**
- An **Invoice** belongs to one **Customer**
- An **InvoiceLine** links one **Invoice** to one **Track** (what was purchased)
- A **Customer** has one support **Employee** via `SupportRepId → Employee.EmployeeId`
- **Employee** has a self-referencing `ReportsTo` column for the org-chart hierarchy

Sample Natural Language Questions

These 20 questions span simple to advanced difficulty. Use them to test your system during development and as part of your evaluation benchmark.

Level	Question
Simple	How many artists are in the database?
Simple	List all genre names.
Simple	What is the most expensive track?
Simple	How many customers are there?
Simple	Show the 5 longest tracks by duration in milliseconds.
Intermediate	Which albums were released by AC/DC?
Intermediate	How many tracks does each genre have? Order by count descending.
Intermediate	List all customers from Brazil.
Intermediate	What are the top 5 best-selling tracks by total quantity sold?
Intermediate	Which employee supports the most customers?
Intermediate	How much total revenue was generated per country?
Intermediate	What are the top 10 artists by number of albums?
Advanced	Which 10 artists have generated the most total revenue?
Advanced	What is the average invoice total per customer?
Advanced	Which customers have spent more than the average customer?
Advanced	What are the top 5 playlists by number of tracks?
Advanced	How many invoices were placed each year? Order by year.
Advanced	Which genres generate the most total revenue?
Edge case	Show all Rock tracks that cost more than \$1.00.
Edge case	Which customers have never placed an order?

System Architecture

Your application must have the following **six core components**. How you connect them is up to you.

#	Component	Responsibility
1	Database Layer	Download the Chinook SQLite file, document the schema, confirm data loads correctly.
2	Schema Context Builder	Extract table names, column names, and key relationships into a compact string that can be injected into the LLM prompt.
3	Prompt Template Engine	A ChatPromptTemplate that includes the schema context, safety rules, and output format instructions alongside the user's question.
4	Text-to-SQL LLM Chain	A LangChain LCEL chain (prompt llm output_parser) that generates a candidate SQL query and a plain-English explanation.
5	Safety Module	Validate that the generated SQL is a pure SELECT statement. Block INSERT, UPDATE, DELETE, DROP, ALTER, CREATE, TRUNCATE, and multi-statement queries.
6	Gradio Interface	A Blocks-based UI with: question input, generated SQL display, explanation text, results DataFrame, and a status/error bar.

Recommended data flow

```
User question → Schema Context Builder → Prompt Template → LLM → Safety Module → Query Executor → Gradio UI
```

Recommended Tech Stack

Library	Role	Notes
LangChain	Orchestration	Chains prompts → LLM → output parsers; model I/O
Gradio	Web UI	Fastest way to build shareable Python ML demos
SQLite	Database	Serverless, file-based SQL — no installation needed
Pandas	Formatting	Turn SQL results into displayable DataFrames
sqlparse	Validation	Detect and block unsafe SQL statement types
Ollama (opt.)	Local LLM	Run llama3, mistral, or codellama on your machine
OpenAI / Groq (opt.)	Cloud LLM	GPT-4o or llama3-70b via API if local is too slow

LangChain concepts used in this project

Concept	Application in This Project
Model I/O	Calling the LLM with a structured prompt and parsing the response
Prompt Templates	ChatPromptTemplate with {schema} and {question} variables
Chains (LCEL)	prompt llm output_parser pipeline
Output Parsers	Extracting clean SQL and explanation from raw LLM text
Memory (stretch)	ConversationBufferMemory for follow-up question support
Data Connections	SQLiteDatabase utility from langchain_community

Recommended Project File Structure

File	Purpose
app.py	Gradio app entry point — launch this to run the demo
sql_engine.py	LangChain Text-to-SQL pipeline and chain definition
prompt_templates.py	All ChatPromptTemplate definitions
db_setup.py	Database setup, download helper, and schema extractor
safety.py	SQL safety validation logic
Chinook_Sqlite.sqlite	The Chinook database file (downloaded automatically)
evaluation.csv	Your 20+ test questions with expected and actual SQL
README.md	Setup and run instructions for your project
report.pdf	Architecture, challenges, failure cases, and improvements

Deliverables Checklist

Every team must submit **all six items** below.

#	Deliverable	Details
1	Working Gradio app	Accepts natural language questions, generates SQL, displays results. Must block all write/destructive SQL.
2	Source code	Clear module structure: app.py, sql_engine.py, prompt_templates.py, db_setup.py, safety.py.
3	Database file + schema doc	The Chinook .sqlite file and a written schema description explaining tables, columns, and relationships.
4	README	Step-by-step setup and run instructions. Anyone should be able to run your app after reading it.
5	Evaluation sheet (CSV)	At least 20 test questions. For each: question, expected SQL, generated SQL, pass/fail, and notes.
6	Short report	Architecture diagram, technical challenges, failure case analysis, and what you would improve. Written or slide-based.

Minimum Functional Requirements

- Accept natural language questions and convert them to SQL.
- Display the generated SQL **before** execution — users must see what will run.
- Execute **only read-only SELECT queries** — all write/destructive SQL must be blocked.
- Display query results in a Gradio DataFrame component.
- Handle invalid, ambiguous, or unanswerable questions with a clear, friendly message.
- Log each question, generated SQL, and execution status (printed or saved to file).

Tips for a Successful Project

- **Build end-to-end first.** Get a working pipeline from question → SQL → results — even if SQL quality is imperfect — before polishing anything else.
- **Invest in your prompt.** The quality of the prompt template is the single biggest factor in SQL correctness. Include the full schema, safety rules, and 2–3 few-shot examples.
- **Test with simple questions first.** Single-table SELECTs should work before you tackle 3-table JOINS.
- **Document failure cases.** Your evaluation report should honestly analyse where and why the system fails — this is worth as much as having a perfect system.
- **Safety is non-negotiable.** A system that can accidentally delete data has failed a core requirement regardless of how good the SQL quality is.
- **A working simple app beats a broken complex one.** Scope carefully.

Useful References

Resource	URL
Chinook Database	github.com/lerocha/chinook-database
LangChain LCEL Docs	python.langchain.com/docs/expression_language/
LangChain SQL Toolkit	python.langchain.com/docs/integrations/tools/sql_database/
Gradio Docs	gradio.app/docs/
Gradio Blocks Guide	gradio.app/guides/blocks-and-event-listeners
Ollama Model Library	ollama.com/library
sqlparse Docs	sqlparse.readthedocs.io/
SQLite SQL Reference	sqlite.org/lang.html

Start early — ask questions early — build something you are proud of. Good luck!